

Unit 5 — Building Data Pipelines & Kafka Streams

Syllabus: building data pipelines · when to use pipelines · when to use Kafka Connect vs producer/consumer · Kafka Streams · stream-processing design patterns · architecture overview · how to choose a stream-processing framework · examples: word count, stock-market statistics, click-stream enrichment

Sources: Shapira et al. *Kafka: The Definitive Guide* 2nd ed. Ch.9 (Building Data Pipelines) and Ch.14 (Stream Processing) + all question papers. **BOOK p.xxx** ↗ opens *Kafka.pdf* at that page. **Note:** *Kafka.pdf* is a reflowed e-book with **no printed page numbers**, so p.xxx = PDF page number (offset 0). Your *Unit5.pdf* extract skips the Kafka Connect internals (pp.340–368) and the stream-processing concepts (pp.598–613); both are filled here from the full book. Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 5 (Q9/Q10 — choose one, 16 marks)

Rank	Topic	Asked in
1	Kafka Connect vs custom producer/consumer ; why Connect; its features	SEE Feb'26 9a (10M), 10b (8M) · CIE-1 Dec'24 5a
2	Considerations when building a data pipeline (fraud-detection example)	CIE-2 Dec'25 Q3 (10M) · Model QP 9a
3	Kafka pipeline for a scenario (logs from microservices)	SEE Feb'26 9b (6M)
4	Joins : stream–table, table–table, stream–stream (windowed)	CIE-2 Dec'25 Q4 (10M) · Model QP 10b
5	Processing with local state (diagram); internal vs external state	SEE Feb'26 10a (8M) · Model QP 1.10
6	Single-event processing pattern (order-confirmation e-mail)	Model QP 10a
7	Connect: connector types, RegexRouter, TimestampRouter, SMTs, "how Connect deals with connectors"	CIE-2 Dec'25 A4, A5 · Model QP 9b
8	Kafka Streams examples (word count, stock stats, click-stream) – "list two examples"	SEE Feb'26 1.10 · syllabus
9	Kafka as pipeline – why (decoupling buffer)	CIE-2 Dec'25 A3

1 · Building data pipelines **BOOK p.327** ↗

- Two common cases: (1) Kafka is **one of the endpoints** (Kafka → S3, MongoDB → Kafka); (2) Kafka is the **intermediary** between two systems (Twitter → Kafka → Elasticsearch). **Kafka Connect was added in version 0.9.**
- Main value of Kafka in pipelines: a very large, reliable buffer between stages** — it **decouples producers and consumers**, lets the same source data feed many targets with different timeliness/availability needs; with reliability, security and efficiency it fits most pipelines. **BOOK p.327** ↗ **IMPORTANT** CIE-2 Dec'25 A3 "Justify Kafka as an effective pipeline"

- Think of the bigger picture (≥ 2 endpoints) — short-term point-to-point integrations create a costly integration mess. [BOOK p.328 ↗](#)

1.1 Considerations when building pipelines [BOOK p.328 ↗](#) **IMPORTANT** CIE-2 Dec'25 Q3 (10M)

· Model QP 9a

#	Consideration	Key points	How Kafka helps
1	Timeliness BOOK p.329 ↗	Some systems want daily bulk, others ms latency; support both and ease migration	Kafka is a giant buffer decoupling time-sensitivity : real-time producers, batch consumers (or vice versa); easy back-pressure (consumers drive rate)
2	Reliability BOOK p.329 ↗	No single point of failure, fast automatic recovery; delivery guarantees — at-least-once (duplicates on retry) or exactly-once	Kafka gives at-least-once itself; exactly-once with transactional/unique-key stores; Connect API helps end-to-end exactly-once via offsets
3	High & varying throughput BOOK p.330 ↗	Scale to very high rates and adapt to sudden spikes	Buffer removes need for complex back-pressure; add producers/consumers independently; compression; hundreds of MB/s on modest clusters; Connect parallelises
4	Data formats BOOK p.331 ↗	Reconcile types (XML/relational in → Avro in Kafka → JSON to ES, Parquet to HDFS, CSV to S3); also pull vs push sources	Kafka and Connect are format-agnostic ; pluggable converters ; schema carried so changes (new MySQL column) can propagate
5	Transformations: ETL vs ELT BOOK p.332 ↗	ETL = transform in the pipeline (saves storage but ties hands of downstream users). ELT = minimal change, raw data to target, transform there (flexible, costs target CPU/storage)	Connect SMTs for light changes; joins/aggregations → Kafka Streams. Don't clean too early — keep raw data
6	Security BOOK p.334 ↗	Who can access ingested data, encryption in transit across data centres, who may change pipelines, authenticating to external stores, PII compliance	Wire encryption, SASL authentication, authorization, audit log; external secret providers (Vault) in Connect
7	Failure handling BOOK p.336 ↗	Plan for bad records: prevent, recover, fix and reprocess, detect late	Kafka stores events long-term so you can go back in time and replay
8	Coupling & agility BOOK p.336 ↗	Avoid ad-hoc pipelines per pair, loss of metadata (no schema evolution), extreme processing	One generic platform; preserve raw data and schemas; let downstream apps decide

 **MNEMONIC — "T-R-H-D-T-S-F-C" → "Tom Reads Huge Data To Stay Fully Current":**
Timeliness · Reliability · High-throughput · Data formats · Transformations · Security · Failure handling · Coupling

HOW TO WRITE IT — CIE-2 Dec'25 Q3 / Model QP 9a — real-time fraud detection in online transactions

Apply each point to fraud: **Timeliness** ms–seconds (decision before card approval); **Reliability** no loss, at-least-once/exactly-once, replicated brokers (acks=all); **Throughput** spikes (sales days), partitions + consumer groups; **Formats** Avro + schema registry, schema evolution as new fraud features added; **Transformations** ELT — keep raw transactions, enrich in Kafka Streams; **Security** TLS, SASL, ACLs, PII masking (MaskField SMT), audit; **Failure handling** dead-letter queue for malformed events, replay; **Coupling** keep source systems decoupled via Kafka. (CIE-2 Dec'25 solution lists: scalability, reliability & fault tolerance, data format & schema evolution, timeliness, ETL vs ELT, access control, failure handling and coupling.)

2 · When to use Kafka Connect vs producer/consumer [BOOK p.338 ↗](#)

IMPORTANT SEE Feb'26 9a, 10b · CIE-1 Dec'24 5a

	Kafka clients (producer/consumer)	Kafka Connect
Use when	You can modify the application's code and want to push/pull data to/from Kafka (clients are embedded in your app)	Connecting Kafka to datastores you did not write and cannot/will not modify
Effort	You write code: data types, configuration, error handling, offsets, scaling, monitoring	Usually just write configuration files — connectors exist for most systems
Built-in features	You build them	Configuration management, offset storage, parallelisation, error handling, support for different data types, standard REST management API
Example	E-commerce app publishes order events; custom sensor service	MySQL/PostgreSQL → Kafka; Kafka → S3/Elasticsearch/data warehouse

- If no connector exists you can write an app with clients **or** use the Connect API; **Connect is recommended** because it supplies those features and is a standard part of the Kafka ecosystem. [BOOK p.339 ↗](#)

When a custom producer/consumer is better (limitations of Connect) **IMPORTANT**

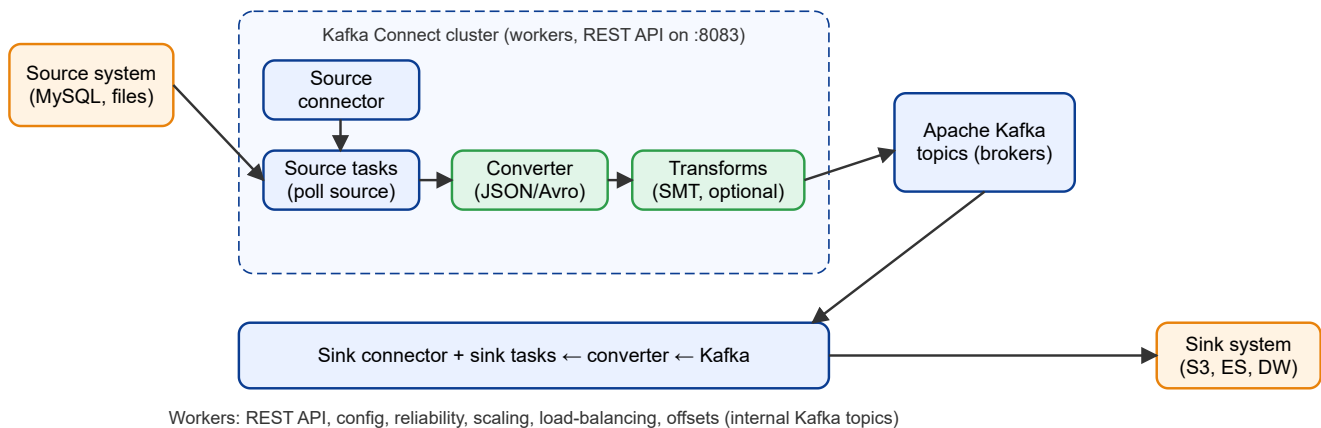
SEE Feb'26 9a(iii), 10b

- No connector exists for a proprietary/odd source **and** you control the source code ⇒ embed a producer directly (no extra moving part).
- Need **complex per-record business logic**, joins or aggregation — SMTs are stateless and single-message; use Kafka Streams/consumer code.
- Need **very low latency / fine control** over batching, partitioning, keys, transactions in application code.
- Source pushes data (events inside your app) rather than being a datastore to be polled; JDBC-style connectors poll tables with timestamps (inefficient/inaccurate) ⇒ use CDC (Debezium) or app-level events.

- Limitations of Connect themselves: extra cluster to operate; connector quality varies (FileStream connectors not for production); polling-based JDBC source adds load and may miss updates. (Our synthesis from pp.338, 347, 352.)

3 · Kafka Connect in detail [BOOK p.339 ↗](#)

Gap in *Unit5.pdf* — read from *Kafka.pdf* pp.339–368.



Kafka Connect architecture (redrawn — our diagram; the book describes it in text only, pp.360–365)

DEFINITION — Kafka Connect

Part of Apache Kafka: a **scalable and reliable way to copy data between Kafka and other datastores**. It provides APIs and a runtime for **connector plug-ins**. It runs as a **cluster of worker processes**; connectors are configured/managed through a **REST API**; connectors start **tasks** to move data in parallel; **converters** store data in Kafka in different formats.

[BOOK p.339 ↗](#)

Concept	Role (pp.360–364)
Connector	Decides how many tasks run, how to split the copying work between them, gets task configs from workers (e.g. JDBC source: min(tasks.max, number of tables)). Two types: Source connector pulls data from an external system into Kafka; Sink connector sends data from Kafka to an external system. IMPORTANT CIE-2 Dec'25 A4
Tasks	Actually move data in/out of Kafka. Source tasks poll the external system and return records; sink tasks receive records from Kafka via the worker and write to the target .
Workers	"Container" processes that run connectors and tasks: handle the REST requests , store connector config in an internal Kafka topic, start tasks, auto-commit offsets , handle retries; if a worker dies others reassign its connectors/tasks (heartbeats via the consumer protocol); new workers get load ⇒ scalability, fault tolerance, load balancing . Connectors/tasks = "moving data"; workers = REST, config, reliability, HA, scaling.
Converters & data model	Connect has its own in-memory Schema + Value objects. Converters (primitive types, byte arrays, strings, Avro, JSON, JSON Schema, Protobuf) turn them into bytes in Kafka and back ⇒ any connector works with any data format . JSON converter can include or omit schema.
Offset management (BOOK p.364 ↗)	Source records carry a logical partition and offset of the source system (file + line; table + timestamp/ID). After Kafka acknowledges, worker stores the offset (internal topic <code>offset.storage.topic</code> ; also <code>config.storage.topic</code> , <code>status.storage.topic</code>) ⇒ restart from last offset. Sink: workers commit Kafka offsets after <code>put()</code> succeeds.

Running Connect ([BOOK p.340 ↗](#))

- Ships with Kafka; run on **separate servers** from brokers for production. Start: `bin/connect-distributed.sh config/connect-distributed.properties`; **standalone mode** (`connect-standalone.sh`) runs everything in one worker — used when a connector must run on a specific machine (e.g. syslog listener). ([BOOK p.344 ↗](#))
- Key worker configs: `bootstrap.servers` (give ≥3), `group.id` (same id = same Connect cluster), `plugin.path` (connector jars), `key.converter` / `value.converter` (default JSON), `rest.host.name` / `rest.port` (REST on **8083**).
- Create a connector by POSTing JSON (name + config: `connector.class`, source/target, topic) to `/connectors`; example **FileStreamSource** → **topic** → **FileStreamSink** (pp.344–347, not for production), and **MySQL** → **Kafka (JDBC source, mode=timestamp, topic.prefix)** → **Elasticsearch (sink, key.ignore=true)** (pp.347–355).
- CDC:** JDBC connector polls tables (timestamp/ID) = inefficient; **change data capture** reads the DB transaction log — use the **Debezium** project. ([BOOK p.352 ↗](#))

Single Message Transformations (SMTs) ([BOOK p.356 ↗](#)) **IMPORTANT** CIE-2 Dec'25 A5

([RegexRouter](#), [TimestampRouter](#))

Stateless per-record changes done inside Connect while copying, often without code (set via `transforms` config). More complex (joins/aggregation) ⇒ stateful Kafka Streams.

SMT	Does
Cast	Change a field's data type
MaskField	Replace a field with null — remove sensitive/PII data
Filter	Drop/include messages matching a condition (topic name, header, tombstone)
Flatten	Nested structure → flat (concatenated field names)
HeaderFrom / InsertHeader	Move/copy fields into header / add a static header (e.g. lineage "MessageSource")
InsertField	Add a field (from metadata like offset, or static)
RegexRouter	Change the destination topic using a regular expression and a replacement string
ReplaceField	Remove or rename a field
TimestampConverter	Change time format of a field (Unix epoch → string)
TimestampRouter	Modify the topic based on the message timestamp — useful in sinks to write to time-partitioned tables

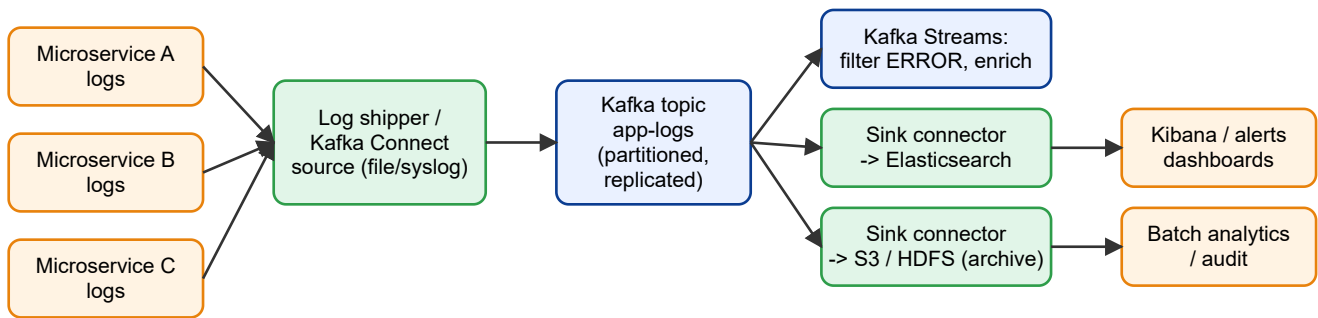
Error handling: `error.tolerance` lets a connector silently drop corrupt messages or route them to a **dead letter queue** topic. ([BOOK p.360](#)) **Alternatives to Connect** ([BOOK p.366](#)): Flume (Hadoop), Logstash/Fluentd (Elasticsearch) when Kafka is just one input; GUI ETL tools (Informatica, NiFi, StreamSets); stream-processing frameworks with built-in connectors (harder to troubleshoot lost/corrupt messages).

3.1 Worked answer — SEE Feb'26 9a (10M): MySQL + PostgreSQL + cloud data warehouse → Kafka **IMPORTANT**

- 1. Why Connect (3M):** sources are **existing datastores whose code you can't/won't change**; ready connectors exist (JDBC/Debezium source, warehouse sink) ⇒ only configuration; built-in config management, offset storage, parallelism, error handling, data-type support, REST API; Kafka decouples MySQL/PostgreSQL from the warehouse and gives timeliness from batch to near-real-time; avoids ad-hoc point-to-point pipelines.
- 2. Scalability & fault tolerance (4M):** distributed workers; connector splits work into `tasks.max` tasks (e.g. one per table); worker failure ⇒ tasks reassigned to other workers; offsets, configs and status stored in internal Kafka topics ⇒ resume after crash; converters + schemas handle schema changes; error tolerance with dead-letter queue; REST API; **CDC with Debezium** for efficient, accurate change capture.
- 3. Two limitations / when custom producer-consumer is better (3M):** (a) complex business logic, joins and aggregations (SMTs stateless) ⇒ Kafka Streams/consumer code; (b) no connector for the system or source you control and need precise control over keys/partitioning/transactions or ultra-low latency; also JDBC polling load and extra cluster to run.

3.2 SEE Feb'26 9b (6M): logs from multiple microservices → Kafka pipeline

IMPORTANT



Kafka pipeline for microservice logs (redrawn — our diagram)

1. Each microservice writes logs; a **source connector / log shipper** (file/syslog, or direct producers) publishes to the **app-logs topic** (partitioned by service/host, replicated).
2. Kafka is the **decoupling buffer**; retention lets late or slow consumers catch up.
3. **Kafka Streams** (single-event filter/map): route ERROR events to a high-priority topic, enrich with metadata (pattern of Fig 14-3).
4. **Sink connectors** to Elasticsearch (search/dashboards, alerts) and S3/HDFS (archive, audit, batch analytics). Use **SMT** TimestampRouter for dated indices, MaskField for PII; dead-letter queue for bad records; TLS/SASL security.

4 · Stream processing — concepts [BOOK p.596 ↗](#)

Gap in *Unit5.pdf* — read from *Kafka.pdf* pp.596–613.

- **Data stream (event stream) = abstraction of an unbounded dataset** — infinite, ever growing. Attributes: **ordered**, **immutable records** (cancellations are new events), **replayable**. [BOOK p.599 ↗](#)
- **Stream processing** = ongoing processing of one or more event streams; a programming paradigm like request-response and batch. [BOOK p.601 ↗](#)

Paradigm	Latency	Mode
Request-response	Lowest (sub-ms to few ms), OLTP	Blocking
Batch processing	High latency/high throughput (minutes–hours), data warehouse/BI	Scheduled
Stream processing	Between the two: continuous	Continuous, non-blocking

Key concepts [BOOK p.603 ↗](#)

Concept	Meaning
Topology BOOK p.603 ↗	Source streams → graph of stream processors (filter, count, group-by, left-join) connected by streams → sink streams. A DAG .
Time BOOK p.604 ↗	Event time (when it occurred — usually what matters), log append (ingestion) time (arrived at broker), processing time (when the app saw it — unreliable, avoid). Kafka Streams uses <code>TimestampExtractor</code> . Standardise time zones.
State BOOK p.607 ↗	Information kept across events (counts, averages, join data). Local/internal state : only that instance, embedded in-memory DB — very fast but limited by memory. External state : external datastore (Cassandra) — unlimited size, shared , but extra latency/complexity/availability issues. IMPORTANT Model QP 1.10 "Internal vs external state"
Stream-table duality BOOK p.609 ↗	Table = current state (mutable); stream = history of changes. Table → stream : capture changes (CDC). Stream → table : apply all changes = materialise . Example: shoe-store inventory (Fig 14-1).
Time windows BOOK p.611 ↗	Choose window size , advance interval (hopping; tumbling when advance = size), grace period (how long it stays updatable for late events); session window = size set by an inactivity gap; aligned to clock or not.
Processing guarantees BOOK p.613 ↗	Exactly-once via Kafka transactions + idempotent producer; enable with <code>processing.guarantee=exactly_once</code> (<code>exactly_once_beta</code> from Streams 2.6 / brokers 2.5 — more efficient).

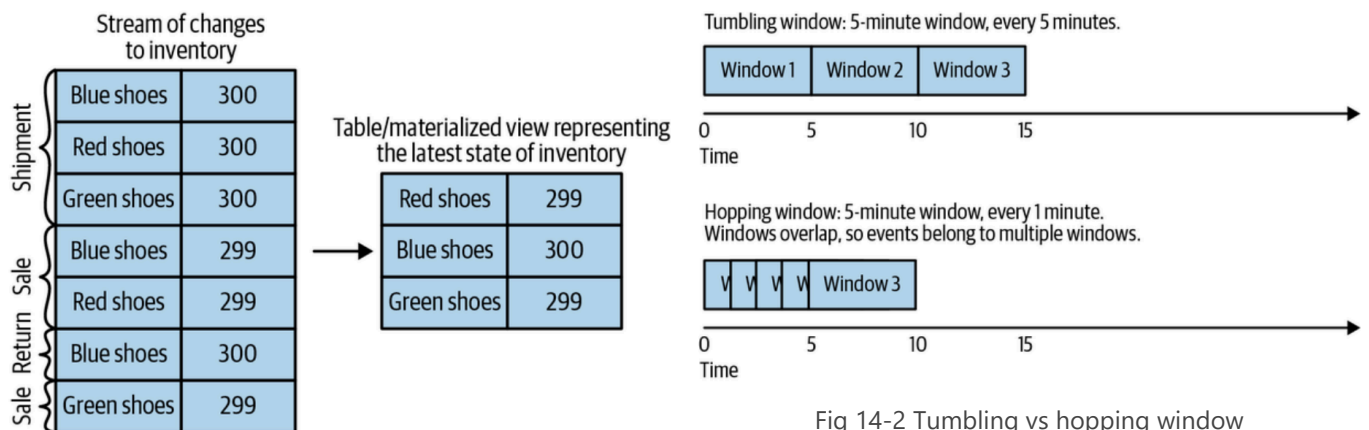


Fig 14-1 Materialising inventory changes (stream → table)

Fig 14-2 Tumbling vs hopping window

5 · Stream-processing design patterns [BOOK p.614 ↗](#)

5.1 Single-event processing (map/filter) [BOOK p.614 ↗](#) **IMPORTANT** [Model QP 10a \(order-confirmation e-mail\)](#)

- Process **each event in isolation**: consume, modify/filter, produce to another stream. **No state** ⇒ recovery/load-balancing trivial (hand the events to another instance). Examples: ERROR log lines → high-priority topic, others → low-priority; JSON → Avro. Can be done with a plain producer + consumer.

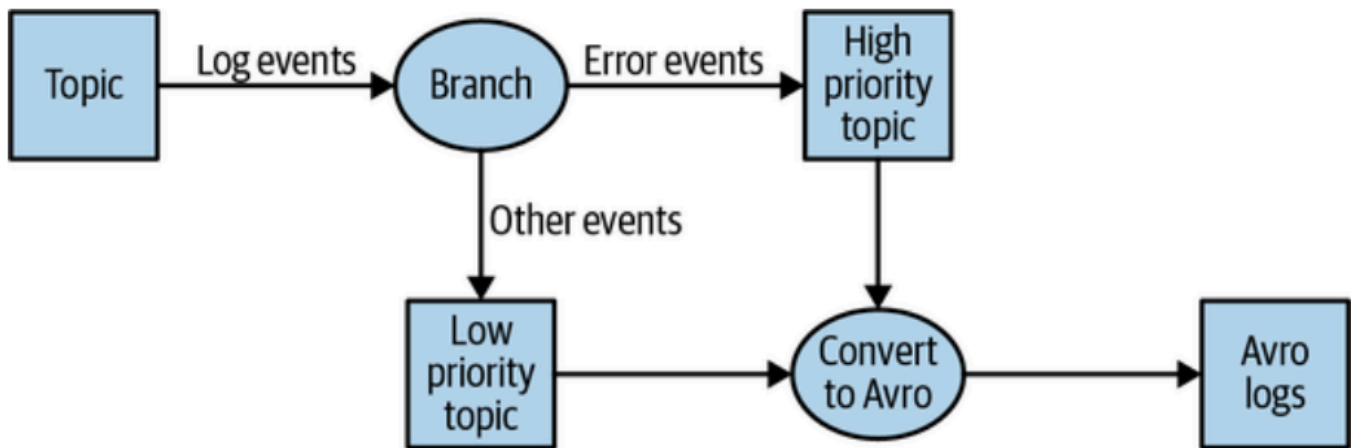


Fig 14-3 Single-event processing topology: Branch → error events to high-priority, others to low-priority → convert to Avro

5.2 Processing with local state BOOK p.615 ↗ **IMPORTANT** SEE Feb'26 10a (8M)

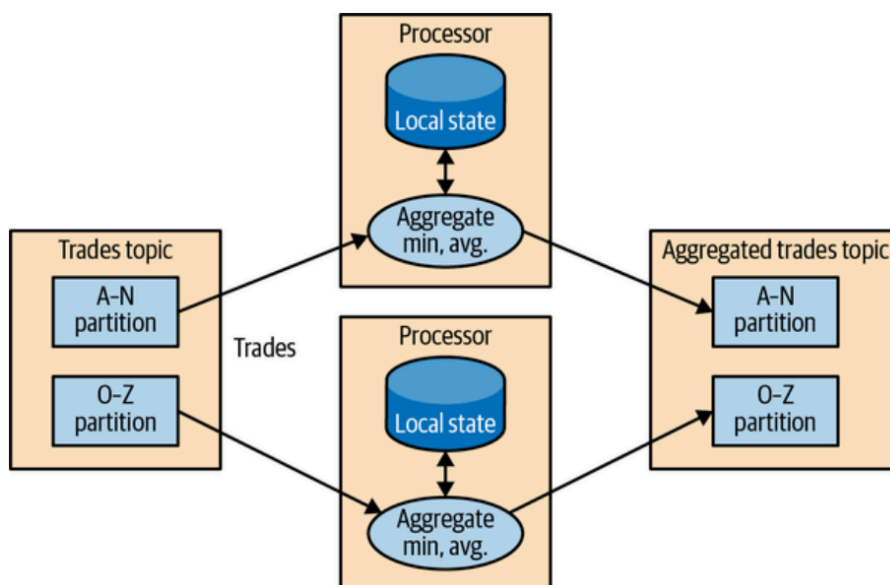


Fig 14-4 Topology for event processing with local state (min/avg price per stock)

- Windowed/group-by aggregates (min and average stock price per day) need state: store minimum, sum and count. Use **local state** because the aggregation is **per key** (stock symbol); a **partitioner** sends all events for one symbol to the same partition ⇒ each instance holds state only for its partitions.
- **Issues:** **Memory usage** (state should fit memory; spilling to disk hurts), **Persistence** (state must survive restarts — Kafka Streams keeps it in embedded **RocksDB** and also sends every change to a **Kafka changelog topic** (log-compacted), so it can be rebuilt by re-reading), **Rebalancing** (when a partition moves, the new owner must recover the right state).

🔥 **HOW TO WRITE IT — SEE Feb'26 10a (8M) "processing with local state — elucidate with diagram"**

Definition (the statement given) → Fig 14-4 → stock min/avg example → partitioner by key ⇒ local state → 3 issues with RocksDB + changelog solution → contrast with external state (§4) → when not enough (needs multiphase, below).

5.3 Multiphase processing / repartitioning BOOK p.618 ↗

- Local state only gives group-by results. For a global result (top 10 stocks of the day) use **two phases**: compute per-symbol daily gain on each instance (local state), write to a **new single-partition topic**, one instance finds the top 10. (MapReduce-like, but all steps in one app.)

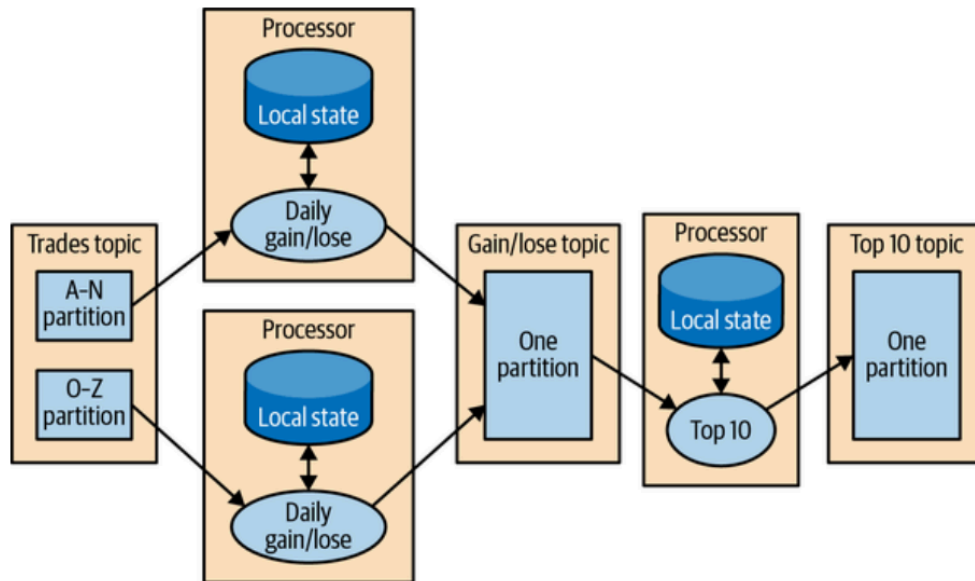


Fig 14-5 Local state + repartitioning steps

5.4 External lookup → stream-table join BOOK p.619 ↗

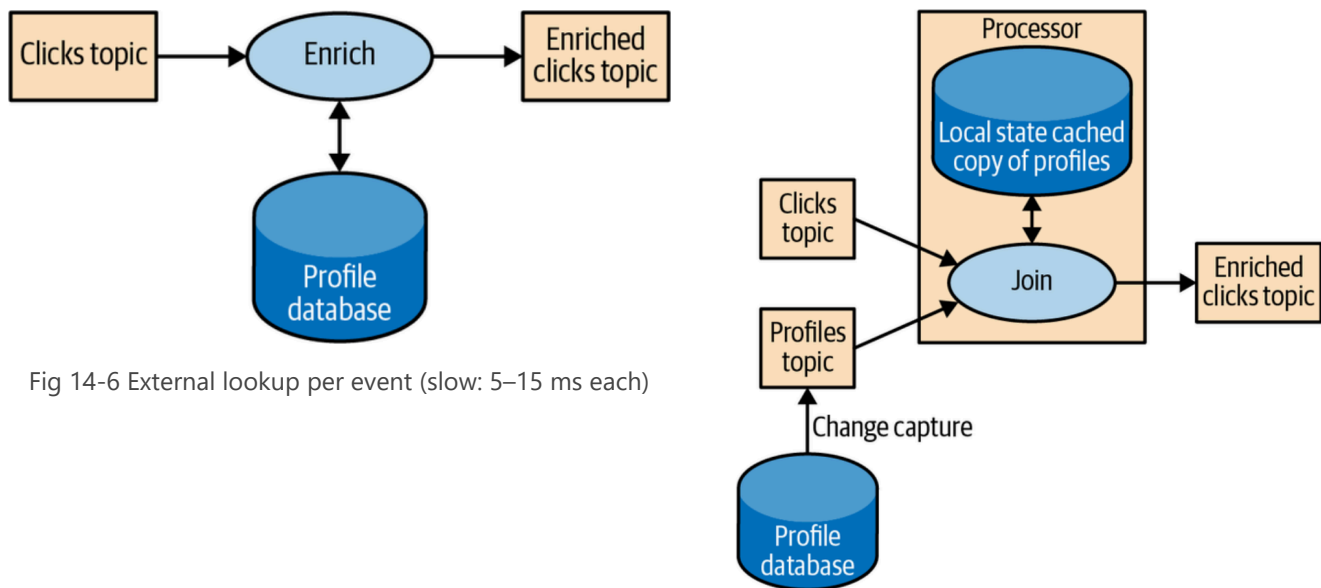


Fig 14-6 External lookup per event (slow: 5–15 ms each)

Fig 14-7 CDC keeps a local cached copy of the table (stream-table join)

- Direct DB lookup per event adds **5–15 ms latency**, overloads the DB (stream processors handle 100K–500K events/s, DB maybe 10K/s) and hurts availability. Caching goes stale. **Solution:** capture DB changes with **CDC** (Kafka Connect/Debezium) into a topic, keep a **local copy of the table in state** and join events against it.

Join	What is joined	Windowed?	Notes
Stream-table (enrichment) BOOK p.619 ↗	Each event of a stream with the current value in a table (materialised via changelog/CDC)	No	Inner, left; e.g. click stream enriched with user profile (like fact table joined with dimension)
Table-table BOOK p.622 ↗	Two materialised tables — current state of both at the time of the operation	No (always non-windowed)	Equi-join : same key, same partitioning ⇒ efficiently distributed; also foreign-key join (key of one joined with any field of other). Inner, left, outer.
Stream-stream (streaming / windowed) BOOK p.623 ↗	Two real event streams — entire history matched by same key within a time window	Yes	e.g. search queries ↔ clicks within seconds; keep a small window of each stream in RocksDB state

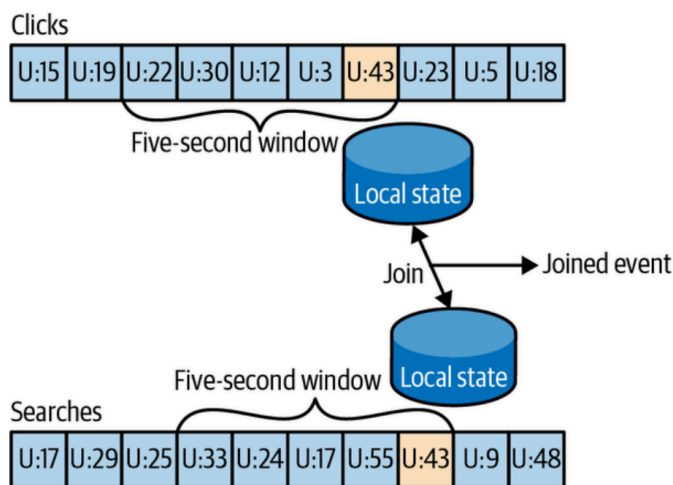


Fig 14-8 Joining two streams: windows on each side (user 43 appears in both)

- Streams in a join must be **partitioned on the same key with the same number of partitions**; Kafka Streams assigns the matching partitions to the **same task**. [BOOK p.624 ↗](#)

5.6 Out-of-sequence events & reprocessing [BOOK p.625 ↗](#)

- Out-of-sequence** (IoT device offline for hours): recognise it (event time older than current), define a **grace period** to reconcile, reconcile **in-band** (no "rerun yesterday's job"), and **update results**. Kafka Streams keeps multiple windows updatable in local state and writes a new aggregate result that replaces the old one (compacted result topic). Longer retention ⇒ more memory.
- Reprocessing**: (1) run the new app version as a **new consumer group** from the first offset, compare results, then switch clients (safer); (2) reset the buggy app (reset tool, clear state/output) — riskier. [BOOK p.627 ↗](#) **Interactive queries** read results straight from the state store. [BOOK p.628 ↗](#)



Fig 14-9 Out-of-sequence events (4:45–4:47 arriving late)

6 · Kafka Streams by example [BOOK p.629 ↗](#) **IMPORTANT** [SEE Feb'26 1.10 · syllabus](#)

- Kafka Streams = library (since 0.10.0) to consume, process and produce events in your own app — **no external framework or cluster**; just start more instances. Two APIs: low-level **Processor API** and high-level **Streams DSL** (`StreamsBuilder` → topology (DAG) → `KafkaStreams` object → `start()`). [BOOK p.629 ↗](#)
- Every app needs an `application.id` (unique per cluster; names internal topics/stores), `bootstrap.servers`, default Serdes (serialisation/deserialisation).

Example	Pattern shown	Key steps
Word count BOOK p.630 ↗	Map/filter + simple aggregate (local state)	Read <code>wordcount-input</code> lines → <code>flatMapValues</code> split on regex → map word as key → filter out "the" → <code>groupByKey</code> → <code>count()</code> → convert Long to String → <code>to("wordcount-output")</code>
Stock-market statistics BOOK p.633 ↗	Window aggregation (windowed local state)	Trades (ticker, ask price, ask size) → <code>groupByKey()</code> → <code>windowedBy(TimeWindows.of(5 s).advanceBy(1 s))</code> → aggregate into a <code>TradeStats</code> object (min price, count, sum) with a named state store (<code>Materialized</code> , <code>trade-aggregates</code>) → compute average → write windowed result to <code>stockstats-output</code> . Stats updated every second: min ask price, number of trades, average ask price per 5-s window.
Click-stream enrichment BOOK p.637 ↗	Joins (stream-table, stream-stream)	Streams: page views, searches; <code>KTable</code> : user profiles. <code>views.leftJoin(profiles)</code> (stream-table, keeps clicks of unknown users) → <code>leftJoin(searches, JoinWindows.of(1 s).before(0 s))</code> (stream-stream: clicks within 1 s after a search) ⇒ 360° view of user activity for recommendations.

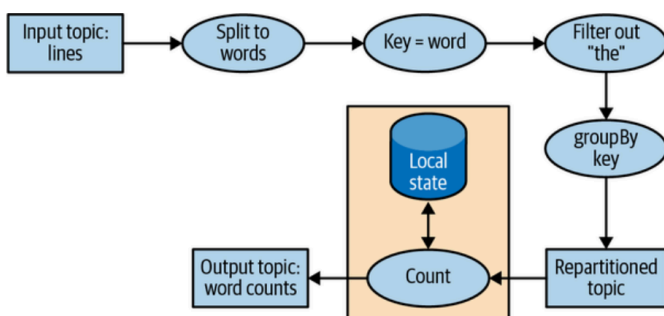


Fig 14-10 Word-count topology (source, split, key=word, filter, groupBy → repartition topic, count with local state, sink)

⚠ CONFLICT / TRAP — Window naming in the stock example

The Kafka book's stock example is a 5-second window advancing every 1 second = **hopping** window in Kafka terms, which Unit 3 (Streaming Data book) calls a **sliding** window. If the question says "window techniques", name it *sliding (hopping)* window and give size 5 s / advance 1 s.

7 · Kafka Streams architecture [BOOK p.641 ↗](#)

- **Topology** = set of operations/transitions each event goes through, made of **processors** (nodes): **source processors** read from a topic, **sink processors** write to a topic. Execution: define logical topology (KStream/KTable DSL) → `build()` creates the physical topology (optimisations possible) → `KafkaStreams.start()`. Test with **TopologyTestDriver** and integration tests (Testcontainers). [BOOK p.641 ↗](#)
- **Scaling** [BOOK p.644 ↗](#): Streams splits the topology into **tasks**; **number of tasks = number of partitions** of the input topics; each task processes its partitions independently with its own local state ⇒ **task = unit of parallelism**. More threads per instance, more instances on more servers; Kafka coordinates. A join puts all needed partitions in the same task ⇒ topics in a join need the same partition count and join-key partitioning. **Repartitioning** (groupBy new key) writes to a new topic and splits the topology into two sub-topologies that run independently.
- **Surviving failures** [BOOK p.649 ↗](#): data is in highly available Kafka; restart resumes from last committed offset; lost local state is rebuilt from the Kafka **changelog**; failed tasks restart on other threads/instances (consumer-group coordination, cooperative rebalancing). Faster recovery: aggressive log compaction, and **standby replicas** that keep state warm.

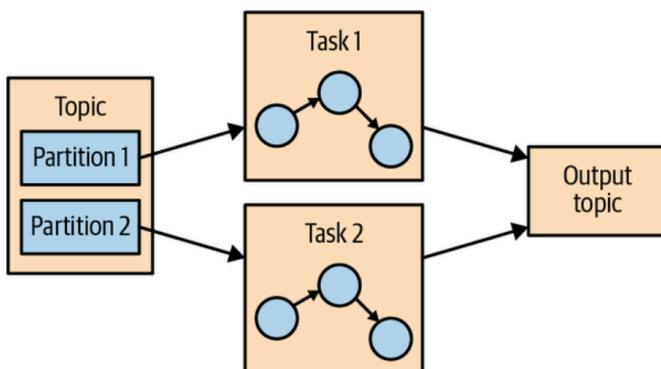


Fig 14-11 Two tasks, one per partition

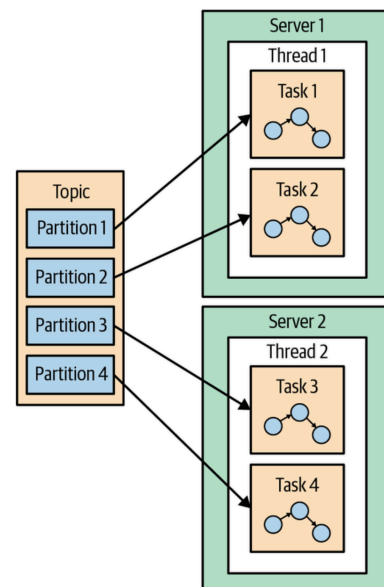


Fig 14-12 Tasks on multiple threads and servers

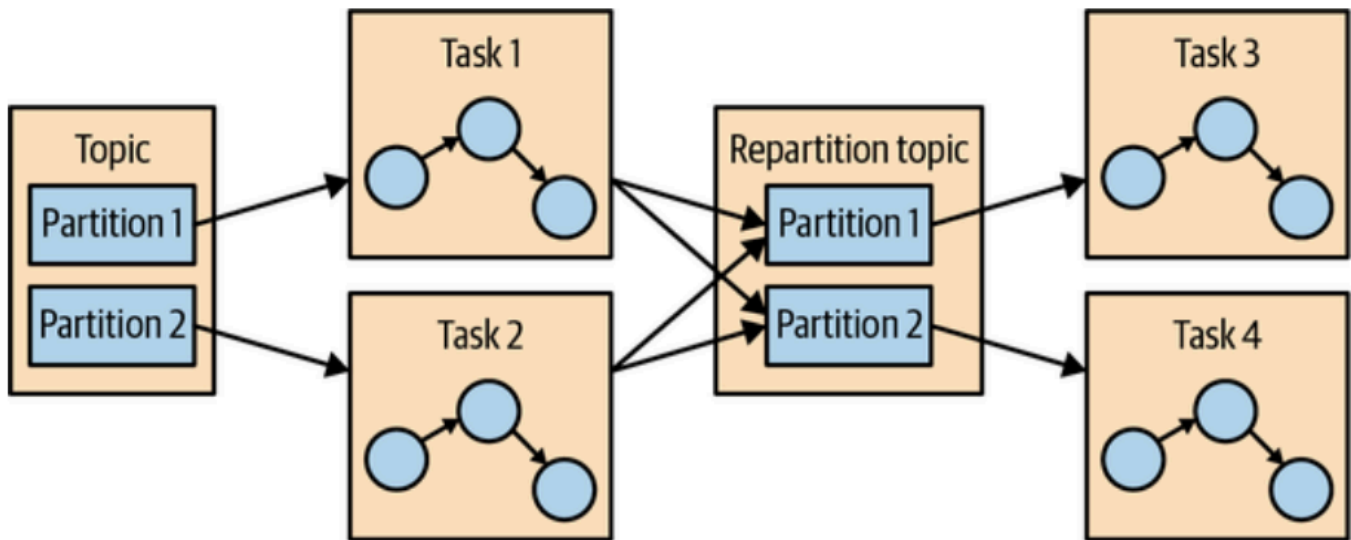


Fig 14-13 Repartition topic separates two sets of tasks

8 · Use cases & how to choose a stream-processing framework

[BOOK p.651 ↗](#)

- **Use cases:** customer service (instant reservation updates), Internet of Things (predictive maintenance), **fraud/anomaly detection** (e.g. beaconing in cybersecurity). [BOOK p.651 ↗](#)

Application type BOOK p.653 ↗	What to look for
Ingest (move data with small changes)	Reconsider — simpler Kafka Connect may do; else need good-quality connectors
Low-millisecond actions	Request-response may be better; else event-by-event low-latency model, not micro-batches
Asynchronous microservices	Integrates with message bus, change-data-capture, good local store as cache/materialised view
Near-real-time analytics	Great local-store support; advanced aggregations, windows, joins

Global criteria: **operability** (deploy, monitor, scale, reprocess), **usability of APIs / ease of debugging**, makes hard things easy, **community**. [BOOK p.655 ↗](#)

9 · Quick revision — Unit 5

Main value of Kafka in a pipeline?	Large reliable buffer decoupling producers and consumers.
Eight pipeline considerations?	Timeliness, reliability, throughput, data formats, transformations (ETL/ELT), security, failure handling, coupling/agility.
ETL vs ELT?	Transform in pipeline vs load raw and transform at target.
When Connect, when clients?	Connect: datastores you can't modify; clients: apps you can modify.
Connect added in Kafka version?	0.9.
Two connector types?	Source (into Kafka), sink (out of Kafka).
Connector vs task vs worker?	Plan/split work; move data; run both + REST/config/HA.
Default Connect REST port?	8083.
Converters available?	Primitive, bytes, string, Avro, JSON, JSON Schema, Protobuf.
Internal topics of Connect?	offset.storage.topic, config.storage.topic, status.storage.topic.
RegexRouter / TimestampRouter?	Change topic by regex / by message timestamp.
MaskField?	Null out sensitive field.
Standalone mode used for?	Connectors that must run on a specific machine.
Better than JDBC polling?	CDC (Debezium) reading the transaction log.
Local vs external state?	Fast but memory-bound vs unlimited but slower.
Where does Streams persist local state?	RocksDB + changelog Kafka topic.
Which join is non-windowed?	Table–table (and stream–table); stream–stream is windowed.
Number of Streams tasks?	= number of input partitions.
Standby replica?	Shadow task keeping state warm for fast failover.
Stock example window?	5 s, advance 1 s (hopping).
Three Kafka Streams examples?	Word count, stock-market statistics, click-stream enrichment.
Enable exactly-once in Streams?	processing.guarantee=exactly_once (or exactly_once_beta).